

OPERATION PRISM

Master Engineering Blueprint

One beam of raw documents in —
a spectrum of verified truth out.

THE FUSION

Lattice (knowledge graph) + Redline (claim verification)
+ Council (adversarial debate) + Glass Box (thought-process tracing)
= the full intelligence analysis cycle as one Spring Boot pipeline

React (Vite) · Spring Boot 3 · MySQL 8 · JWT · RAG · Docker · Cloud

Deterministic Java core governing a probabilistic LLM layer,
with a human verifier as the final authority.

Contents

1	Mission Brief and Fusion Rationale	2
2	System Overview: The Four-Stage Pipeline	2
3	Emergent Capabilities of the Fusion	3
4	Syllabus Alignment Doctrine	3
5	Technology Stack	4
6	High-Level Architecture	4
7	Database Schema	5
8	The Deterministic Engines	8
9	The LLM Layer	14
10	Frontend Design	16
11	API Catalog	17
12	Core Workflows	18
13	Glass Box: The Thought-Process Visualizer	19
14	Security Design	23
15	Performance Layer	24
16	Testing Strategy	25
17	Docker and Deployment	25
18	Build Plan and Phased Safety Net	26
19	Risk Register	27
20	What You Are Actually Building	27
21	References	27

1. Mission Brief and Fusion Rationale

MISSION BRIEF — PRISM

Upload a pile of unstructured documents. PRISM automatically decomposes them into entities, relations, and factual claims; verifies every claim against the rest of the corpus with a deterministic rule engine plus LLM judgment; detects contradictions and turns them into a self-generated debate agenda where three LLM analysts argue with citations while one of them is armed with the verification engine; and synthesizes a final report in which every sentence is traceable to a passage, a verdict, or a graph edge. The Glass Box module records and replays the entire thought process of the machine.

Demo kill shot: upload 30 memos → org chart blooms with confidence-colored edges → three contradictions auto-detected → Hawk/Dove/Skeptic debate live, the Skeptic demolishes a claim with a machine verdict badge → you chair, you weight, the report writes itself → press Replay and watch the machine's reasoning animate on screen.

Syllabus coverage: 10/10 **Wow:** 5/5 **Collision risk:** none **Build window:** ~14 weeks

Why these three systems are one system

Lattice, Redline, and Council are not three projects — they are three stages of a single pipeline. Lattice *structures* the raw world (understand). Redline *tests* every assertion against evidence (verify). Council *argues* over what remains uncertain (judge). Merged, they form the complete intelligence analysis cycle — collection, processing, verification, analysis, judgment — as one machine.

The engineering invariant

Deterministic Java owns all state; the LLM only proposes. Every LLM output is parsed into validated DTOs, stored as a proposal, and either rule-processed or human-approved before it becomes knowledge.

This invariant is what makes the system testable (deterministic engines admit exact-value assertions), honest (failures are visible, nothing unvalidated becomes knowledge), and defensible in a viva: it is the architecture pattern behind production AI observability and governance platforms.

2. System Overview: The Four-Stage Pipeline

RAW DOCUMENTS (paste / upload)

|

v

```
+-----+
| STAGE 1 -- DECOMPOSE (Lattice) |
| chunking -> LLM extraction -> strict JSON -> @Valid DTOs |
| -> human approval queue -> entities + triples |
+-----+
```

|

v

```
+-----+
| STAGE 2 -- VERIFY (Redline) |
| FULLTEXT retrieval -> rule engine -> LLM judge -> |
| fusion (confidence - hedging penalty, rule override) |
| -> verdicts + cited passages + contradictions |
+-----+
```

|

v

```
+-----+
| STAGE 3 -- JUDGE (Council) |
+-----+
```

```

| contradiction agenda (self-generated) -> debate rounds |
| Hawk || Dove || Skeptic (parallel) -> Skeptic armed with |
| live verdicts + graph metrics -> chair weights (human) |
+-----+
|
| v
+-----+
| STAGE 4 -- SYNTHESIZE + CHAT |
| synthesis report (cited, calibrated confidence) + |
| grounded chat over documents, triples, and verdicts |
+-----+

```

Every stage is independently demoable and consumes the previous stage's output. One retrieval layer (the chunk store with a FULLTEXT index) serves all three consumers: verification RAG, persona-scoped debate retrieval, and user chat.

3. Emergent Capabilities of the Fusion

Three capabilities exist *only* in the combination — this is the section that wins the viva.

1. The confidence-colored knowledge graph

Every graph edge carries a verification status: **green** = supported by two or more documents, **amber** = single source, **red** = contradicted. PageRank is then computable twice — on the full graph and on the verified-only graph — and the two answers differ. *The network looks completely different when you only trust what is verified.*

2. The Skeptic with teeth

When Hawk argues that entity X controls the network, the Skeptic does not merely argue back — it calls the actual verification engine (retrieval, rules, LLM judgment) and the actual graph engine (PageRank, communities), and injects those deterministic results into its prompt. Its argument then contains lines such as: *Claim 3 is UNSUPPORTED — verdict engine, two passages checked, confidence 0.18. PageRank of X is 0.31; the top node is actually Y at 0.58.* An LLM argument backed by deterministic evidence.

3. The self-generating debate agenda

The contradiction detector finds what the corpus cannot settle: relation-level conflicts (document 3 says A reports to B; document 11 says A reports to C), claim-level conflicts (same subject, opposing polarity), and verdict-level open questions (claims stuck at UNSUPPORTED). These become the debate agenda automatically. The system generates its own questions; the human merely chairs.

4. Syllabus Alignment Doctrine

Every unit of the ten-unit training syllabus maps to a concrete, shipped milestone of PRISM. The full contract with acceptance checks appears in the Build Plan (Section 18); the summary:

Unit	Syllabus focus	PRISM milestone
1	Vite, components, MVC, REST, JSON	Scaffold, dashboard, first JSON endpoint
2	Tables, fetch, CRUD, DI, MySQL	Documents CRUD, layered backend
3	Forms, @Valid, DTOs, exceptions	Registration + validation + error UI
4	JPA mappings, pagination, modals	Relation feast: triples, claims, verdicts, graph view
5	Auth context, roles, protected routes	ADMIN / ANALYST / VERIFIER role system
6	JWT, filter chain, @PreAuthorize	Full lockdown, interceptor, role gates
7	Chat UI, WebClient, RAG	Extraction + verification + contradiction + grounded chat
8	Caching, async, logging, metrics	Async pipeline, parallel personas, SSE, retry
9	JUnit, Mockito, Docker	Exact-value engine tests, Testcontainers, compose
10	Deploy, Swagger, README	Render/Railway + Vercel, Swagger, 4-minute demo

Honest scope statement

This is a 5/5-difficulty build: roughly 1.5–2× any single component project. The design absorbs the three hard problems (extraction reliability, claim decomposition, agent coordination) via the approval queue, strict JSON schemas with retries, and phased rollout — but does not delete them. The phased safety net in Section 18 is the risk management.

5. Technology Stack

Layer	Technology	Why this exact one
Language	Java 17+	LTS; matches Spring Boot 3.x training env
Backend	Spring Boot 3.3.x (MVC)	Syllabus core; layered architecture
Data	Spring Data JPA + Hibernate	Unit 4 mappings
Database	MySQL 8 + FULLTEXT on chunks	Syllabus core + free-text retrieval (RAG v1)
Security	Spring Security + JWT 0.12.x	Units 5–6; filter chain + <code>@PreAuthorize</code>
LLM client	WebClient (spring-webflux dependency only)	Non-blocking calls, syllabus-listed
Retry	Spring Retry	Clean <code>@Retryable</code> on LLM calls
Cache	Caffeine via starter-cache	In-JVM <code>@Cacheable</code> , zero extra infra
Async	<code>@EnableAsync</code> + two custom executors	Pipeline vs LLM thread pools
Streaming	<code>SseEmitter</code> (MVC-native)	Debate chamber + live traces
Docs	springdoc-openapi	Swagger UI, Unit 10
Frontend	React 18 + Vite + React Router v6	Syllabus core
Graph viz	Cytoscape.js	Canvas rendering, edge styling, free
Markdown	react-markdown	Chat + synthesis rendering
Tests	JUnit 5, Mockito, Testcontainers, MockMvc	Real-MySQL integration tests
Dev LLM	Ollama (OpenAI-compatible endpoint)	Free local development
Prod LLM	Any OpenAI-compatible API	Demo quality
Deploy	Render/Railway + Vercel/Netlify	Unit 10 split

LLM cost control (baked into configuration): max 12 claims per document, max 3 debate rounds, 512-token persona cap, cheap model for extraction; only debate and synthesis use the best model.

6. High-Level Architecture

```

REACT (Vite SPA)
|-- Public: Login / Register
|-- ANALYST: Documents, Upload, Pipeline monitor, Claims, Graph, Chat
|-- VERIFIER: Approval Queue, Verdict adjudication, Debate Chamber (Chair)
|-- ADMIN: User management, Actuator metrics
    | axios + JWT interceptor
    v
SPRING BOOT (MVC, layered)
|-- web/      controllers (thin, DTO in/out, @PreAuthorize)
|-- service/  business logic, transactions
|-- engine/   * pure Java: GraphEngine, ClaimRuleEngine,
    | ContradictionDetector, DebateEngine (FSM)
|-- llm/      LlmClient (WebClient + Retry), PromptLibrary, JsonExtractor
|-- repo/     Spring Data JPA
|-- trace/    * TraceRecorder + TraceEventBus (Glass Box)
|-- async/    pipeline + llm + trace executors
    | JPA / Hibernate
    v
MySQL 8      (19 tables, FULLTEXT index on chunks)

```

Layering rule: controllers never touch engines directly — services orchestrate **engine** (deterministic), **llm** (probabilistic), and **repo** (state). Engines carry no Spring dependencies where possible: pure, static-injectable classes, which is what makes them unit-test heaven.

Package tree:

```
com.prism
|-- PrismApplication.java
|-- config/      SecurityConfig, JwtFilter, AsyncConfig, CacheConfig,
|                WebClientConfig, OpenApiConfig, CorsConfig
|-- auth/        AuthController, AuthService, JwtService, dtos/
|-- user/        UserController, UserService, Role (enum), User
|-- document/    DocumentController, DocumentService, Chunker,
|                Document, DocumentChunk, repos/, dtos/
|-- extraction/  ExtractionService, ExtractionEngine *,
|                LlmClient, PromptLibrary, ExtractionResultDto
|-- knowledge/   EntityService (normalize + dedup), TripleService,
|                ApprovalController, Entity, Triple, repos/
|-- graph/       GraphController, GraphService, GraphEngine *
|-- claims/      ClaimController, ClaimService, VerificationEngine *,
|                ClaimRuleEngine *, Claim, Verdict, repos/
|-- contradiction/ ContradictionController, ContradictionDetector *
|-- debate/      DebateController, DebateEngine * (FSM),
|                PersonaAgent, SkepticAnalyst *, DebateEventBus,
|                Debate, Round, Argument, SynthesisReport
|-- chat/        ChatController, RagService, PromptLibrary
|-- trace/       TraceRecorder *, TraceEventBus, TraceRun, TraceStep *
|-- common/      GlobalExceptionHandler, PageResponse<T>, ApiError
```

7. Database Schema

Nineteen tables. All code below is MySQL 8 DDL.

```
-- ===== AUTH =====
CREATE TABLE users (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  username    VARCHAR(50) NOT NULL UNIQUE,
  email       VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,          -- BCrypt
  role        ENUM('ADMIN','ANALYST','VERIFIER') NOT NULL DEFAULT 'ANALYST',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ===== STAGE 1: DOCUMENTS & CHUNKS =====
CREATE TABLE documents (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  uploader_id BIGINT NOT NULL,
  title       VARCHAR(200) NOT NULL,
  content_text MEDIUMTEXT NOT NULL,
  status       ENUM('UPLOADED','CHUNKED','EXTRACTING',
                    'AWAITING_APPROVAL','READY') DEFAULT 'UPLOADED',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (uploader_id) REFERENCES users(id)
);

CREATE TABLE document_chunks (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  document_id BIGINT NOT NULL,
  chunk_index INT NOT NULL,
  content     TEXT NOT NULL,
  FOREIGN KEY (document_id) REFERENCES documents(id),
  FULLTEXT INDEX ft_chunk_content (content)      -- RAG retrieval
);

CREATE TABLE extraction_runs (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  document_id BIGINT NOT NULL,
  status       ENUM('RUNNING','DONE','FAILED') DEFAULT 'RUNNING',
  -- pipeline observability
```

```

model_used    VARCHAR(80),
triples_out   INT DEFAULT 0,
claims_out    INT DEFAULT 0,
error_msg     VARCHAR(500),
started_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
finished_at   TIMESTAMP NULL,
FOREIGN KEY (document_id) REFERENCES documents(id)
);

-- ===== STAGE 1: ENTITIES & TRIPLES (Lattice) =====
CREATE TABLE entities (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  raw_name      VARCHAR(160) NOT NULL,
  normalized_name VARCHAR(160) NOT NULL,          -- dedup key
  type          ENUM('PERSON','ORG','LOCATION','OTHER') DEFAULT 'OTHER',
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_norm (normalized_name, type)
);

CREATE TABLE entity_profiles (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  entity_id     BIGINT NOT NULL UNIQUE,
  display_name  VARCHAR(160),
  description   TEXT,
  FOREIGN KEY (entity_id) REFERENCES entities(id)
);

CREATE TABLE triples (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  run_id        BIGINT NOT NULL,
  subject_entity_id BIGINT NOT NULL,
  predicate     VARCHAR(60) NOT NULL,
  object_entity_id BIGINT NOT NULL,
  source_document_id BIGINT NOT NULL,
  source_sentence TEXT NOT NULL,
  status        ENUM('PENDING','APPROVED','REJECTED') DEFAULT 'PENDING',
  verified_by   BIGINT NULL,
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  decided_at    TIMESTAMP NULL,
  FOREIGN KEY (run_id) REFERENCES extraction_runs(id),
  FOREIGN KEY (subject_entity_id) REFERENCES entities(id),
  FOREIGN KEY (object_entity_id) REFERENCES entities(id),
  FOREIGN KEY (source_document_id) REFERENCES documents(id),
  FOREIGN KEY (verified_by) REFERENCES users(id),
  INDEX idx_sp (subject_entity_id, predicate)
);

-- ===== STAGE 2: CLAIMS & VERDICTS (Redline) =====
CREATE TABLE claims (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  run_id        BIGINT NOT NULL,
  source_document_id BIGINT NOT NULL,
  source_sentence TEXT NOT NULL,
  subject_entity_id BIGINT NOT NULL,
  claim_text    TEXT NOT NULL,
  polarity      ENUM('POSITIVE','NEGATIVE','NEUTRAL') DEFAULT 'NEUTRAL',
  status        ENUM('NEW','JUDGED','CONTESTED','ADJUDICATED')
                DEFAULT 'NEW',
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (run_id) REFERENCES extraction_runs(id),
  FOREIGN KEY (source_document_id) REFERENCES documents(id),
  FOREIGN KEY (subject_entity_id) REFERENCES entities(id)
);

CREATE TABLE verdicts (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  claim_id      BIGINT NOT NULL,
  verdict       ENUM('SUPPORTED','UNSUPPORTED','EXAGGERATED',
                    'SOURCE_MISSING') NOT NULL,
  confidence    DECIMAL(4,3) NOT NULL,
  reasoning     TEXT,
  decided_by    ENUM('ENGINE','REVIEWER') NOT NULL DEFAULT 'ENGINE',
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (claim_id) REFERENCES claims(id)
);

CREATE TABLE verdict_passages (
  -- verdict <-> evidence

```

```

    verdict_id BIGINT NOT NULL,
    chunk_id   BIGINT NOT NULL,
    relevance  DECIMAL(4,3),
    PRIMARY KEY (verdict_id, chunk_id),
    FOREIGN KEY (verdict_id) REFERENCES verdicts(id),
    FOREIGN KEY (chunk_id)   REFERENCES document_chunks(id)
);

-- ===== STAGE 3: CONTRADICTIONS & DEBATES (Council) =====
CREATE TABLE contradictions (
    id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    kind          ENUM('RELATION','CLAIM') NOT NULL,
    description   VARCHAR(400) NOT NULL,
    triple_a_id   BIGINT NULL,
    triple_b_id   BIGINT NULL,
    claim_a_id    BIGINT NULL,
    claim_b_id    BIGINT NULL,
    status        ENUM('OPEN','IN_DEBATE','RESOLVED','SUPERSEDED')
                DEFAULT 'OPEN',
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (triple_a_id) REFERENCES triples(id),
    FOREIGN KEY (triple_b_id) REFERENCES triples(id),
    FOREIGN KEY (claim_a_id) REFERENCES claims(id),
    FOREIGN KEY (claim_b_id) REFERENCES claims(id)
);

CREATE TABLE debates (
    id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    contradiction_id BIGINT NOT NULL,
    chair_user_id BIGINT NOT NULL,
    state         ENUM('CREATED','ROUND_ACTIVE','AWAITING_CHAIR',
                    'SYNTHESIZING','COMPLETED','ABORTED')
                DEFAULT 'CREATED',
    current_round INT DEFAULT 0,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at  TIMESTAMP NULL,
    FOREIGN KEY (contradiction_id) REFERENCES contradictions(id),
    FOREIGN KEY (chair_user_id)   REFERENCES users(id)
);

CREATE TABLE debate_rounds (
    id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    debate_id     BIGINT NOT NULL,
    round_number  INT NOT NULL,
    topic        VARCHAR(400),
    started_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at  TIMESTAMP NULL,
    FOREIGN KEY (debate_id) REFERENCES debates(id)
);

CREATE TABLE arguments (
    id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    round_id     BIGINT NOT NULL,
    persona      ENUM('HAWK','DOVE','SKEPTIC') NOT NULL,
    content      TEXT NOT NULL,
    chair_weight  TINYINT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (round_id) REFERENCES debate_rounds(id)
);

CREATE TABLE argument_citations (
    argument_id BIGINT NOT NULL,
    chunk_id    BIGINT NULL,
    triple_id   BIGINT NULL,
    PRIMARY KEY (argument_id, chunk_id, triple_id),
    FOREIGN KEY (argument_id) REFERENCES arguments(id),
    FOREIGN KEY (chunk_id)   REFERENCES document_chunks(id),
    FOREIGN KEY (triple_id)  REFERENCES triples(id)
);

CREATE TABLE synthesis_reports (
    id            BIGINT AUTO_INCREMENT PRIMARY KEY,
    debate_id     BIGINT NOT NULL UNIQUE,
    content      MEDIUMTEXT NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (debate_id) REFERENCES debates(id)
);

```

```
-- ===== STAGE 4: GROUNDED CHAT =====
CREATE TABLE chat_sessions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  title       VARCHAR(150),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE chat_messages (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  session_id  BIGINT NOT NULL,
  role        ENUM('USER','ASSISTANT') NOT NULL,
  content     TEXT NOT NULL,
  cited_sources JSON NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (session_id) REFERENCES chat_sessions(id)
);

-- ===== GLASS BOX: TRACES =====
CREATE TABLE trace_runs (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  kind        ENUM('PIPELINE','VERIFICATION','DEBATE','CHAT',
                  'GRAPH_METRICS') NOT NULL,
  ref_id      BIGINT NOT NULL,
  status      ENUM('RUNNING','DONE','FAILED') DEFAULT 'RUNNING',
  summary     VARCHAR(300),
  total_llm_calls INT DEFAULT 0,
  total_tokens INT DEFAULT 0,
  started_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  finished_at TIMESTAMP NULL,
  INDEX idx_kind_ref (kind, ref_id)
);

CREATE TABLE trace_steps (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  run_id      BIGINT NOT NULL,
  parent_id   BIGINT NULL,                -- self-referencing tree
  seq         INT NOT NULL,
  actor       ENUM('ENGINE','LLM','HUMAN') NOT NULL,
  step_type   VARCHAR(40) NOT NULL,
  title       VARCHAR(200) NOT NULL,
  detail      JSON NULL,
  status      ENUM('OK','QUARANTINED','RETRY','ERROR') DEFAULT 'OK',
  started_at  TIMESTAMP(3) NULL,
  duration_ms INT NULL,
  tokens_used INT NULL,
  FOREIGN KEY (run_id) REFERENCES trace_runs(id),
  FOREIGN KEY (parent_id) REFERENCES trace_steps(id),
  INDEX idx_run_seq (run_id, seq)
);
```

JPA mapping audit (Unit 4)

Mapping	Where it lives
@OneToOne	entity ↔ profile; debate ↔ synthesis
@OneToMany / @ManyToOne	document → chunks; run → triples/claims; debate → rounds → arguments; verdict → claim
@ManyToMany	verdict ↔ passages; argument ↔ citations
Self-referencing	trace step → parent step (the trace tree)
@ManyToMany	

Five mapping shapes, every one domain-driven, zero decoration.

8. The Deterministic Engines

GraphEngine — pure Java, deterministic

```
// engine/GraphEngine.java -- NO Spring annotations. Pure. Testable.
public class GraphEngine {
```

```

public record Edge(Long from, Long to, String predicate,
                  Set<Long> docIds, EdgeConfidence confidence) {}

public enum EdgeConfidence { VERIFIED, SINGLE_SOURCE, CONTRADICTED }

/** PageRank by power iteration. Damping 0.85, tol 1e-6. */
public Map<Long, Double> pageRank(Collection<Long> nodes,
                                  Collection<Edge> edges,
                                  double damping, int maxIter,
                                  double tol) {
    int n = nodes.size();
    Map<Long, Double> rank = new HashMap<>();
    Map<Long, List<Edge>> inEdges = edges.stream()
        .collect(Collectors.groupingBy(Edge::to));
    Map<Long, Long> outDeg = edges.stream()
        .collect(Collectors.groupingBy(Edge::from,
            Collectors.counting()));
    nodes.forEach(id -> rank.put(id, 1.0 / n));

    for (int it = 0; it < maxIter; it++) {
        double dangling = nodes.stream()
            .filter(id -> !outDeg.containsKey(id))
            .mapToDouble(rank::get).sum();
        Map<Long, Double> next = new HashMap<>();
        for (Long id : nodes) {
            double inbound = inEdges
                .getOrDefault(id, List.of()).stream()
                .mapToDouble(e -> rank.get(e.from())
                    / outDeg.get(e.from()))
                .sum();
            next.put(id, (1 - damping) / n
                + damping * (inbound + dangling / n));
        }
        double delta = 0;
        for (Long id : nodes)
            delta += Math.abs(next.get(id) - rank.get(id));
        rank = next;
        if (delta < tol) break;          // converged
    }
    return rank;
}

/** Edge confidence: green if 2+ docs, amber if 1, red if
    contradicted. */
public EdgeConfidence confidence(Set<Long> docIds,
                                boolean contradicted) {
    if (contradicted) return EdgeConfidence.CONTRADICTED;
    if (docIds.size() >= 2) return EdgeConfidence.VERIFIED;
    return EdgeConfidence.SINGLE_SOURCE;
}

/** Label propagation community detection. */
public Map<Long, Integer> communities(Collection<Long> nodes,
                                       Collection<Edge> edges,
                                       int maxRounds) {
    Map<Long, Integer> label = new HashMap<>();
    nodes.forEach(id -> label.put(
        id, Math.toIntExact(id % 100000)));
    Map<Long, List<Edge>> adjacency = new HashMap<>();
    edges.forEach(e -> {
        adjacency.computeIfAbsent(e.from(),
            k -> new ArrayList<>()).add(e);
        adjacency.computeIfAbsent(e.to(),
            k -> new ArrayList<>()).add(e);
    });
}

```

```

    for (int r = 0; r < maxRounds; r++) {
        Map<Long, Integer> next = new HashMap<>(label);
        for (Long id : nodes) {
            next.put(id, adjacency
                .getOrDefault(id, List.of()).stream()
                .map(e -> label.get(e.to().equals(id)
                    ? e.from() : e.to()))
                .collect(Collectors.groupingBy(
                    Function.identity(), Collectors.counting()))
                .entrySet().stream()
                .max(Map.Entry.comparingByValue())
                .map(Map.Entry::getKey)
                .orElse(label.get(id)));
        }
        if (next.equals(label)) break;
        label = next;
    }
    return label;
}
}

```

The killer feature this enables: compute PageRank on all edges vs. verified-only edges and compare. Two different answers, one button.

ClaimRuleEngine — deterministic forensic linguistics

```

// engine/ClaimRuleEngine.java -- pure functions, table-testable
@Component
public class ClaimRuleEngine {

    private static final List<String> WEASEL = List.of(
        "reportedly", "allegedly", "it is believed", "sources say",
        "some say", "it is thought", "appears to", "seems to",
        "unconfirmed", "rumored");

    private static final List<String> ABSOLUTES = List.of(
        "always", "never", "everyone", "no one", "all ",
        "impossible", "certainly", "definitely", "guaranteed");

    public record RuleAssessment(int weaselHits, boolean absolutes,
        List<String> words,
        double hedgingPenalty) {}

    public RuleAssessment assess(String claimText) {
        String t = " " + claimText.toLowerCase() + " ";
        List<String> hits = WEASEL.stream()
            .filter(t::contains).toList();
        boolean abs = ABSOLUTES.stream().anyMatch(t::contains);
        double penalty = Math.min(
            0.15 * hits.size() + (abs ? 0.10 : 0), 0.60);
        return new RuleAssessment(
            hits.size(), abs, hits, penalty);
    }
}

```

VerificationEngine — the Redline core

```

@Service
public class VerificationEngine {

    private final ClaimRuleEngine rules;
    private final LlmClient llm;
    private final ChunkRepository chunks;    // FULLTEXT query

    public Verdict verify(Claim claim) {

```

```

// 1. Retrieve evidence (MySQL FULLTEXT, top-k)
List<DocumentChunk> evidence =
    retrieve(claim.getClaimText(), 5);

// 2. Deterministic rules run FIRST -- they gate the LLM
RuleAssessment ra = rules.assess(claim.getClaimText());

// 3. SOURCE_MISSING short-circuit: no LLM call, no cost
if (evidence.isEmpty()) {
    return Verdict.of(claim, SOURCE_MISSING, 0.9,
        "No corpus passage matched this claim.",
        List.of());
}

// 4. LLM judge with retrieved context (retry-wrapped)
JudgeResponse jr = llm.judge(
    PromptLibrary.judge(claim, evidence));

// 5. Fuse: LLM confidence - deterministic hedging penalty
double fused = clamp(
    jr.confidence() - ra.hedgingPenalty(), 0, 1);
if (jr.verdict() == SUPPORTED && ra.absolutes()
    && ra.weaselHits() >= 2)
    jr = jr.demoteTo(EXAGGERATED); // rule override

return Verdict.of(claim, jr.verdict(), fused,
    jr.reasoning(), evidence);
}
}

```

Design principle on display: rules gate the model, then rules can overrule the model. The deterministic layer has veto power over the AI — point at `jr.demoteTo(EXAGGERATED)` in the viva and say exactly that.

ContradictionDetector — the self-generating agenda

```

@Service
public class ContradictionDetector {

    public List<Contradiction> detectRelationLevel(
        List<Triple> approved) {
        record SP(Long subject, String predicate) {}
        Map<SP, Map<Long, List<Triple>>>> grouped =
            approved.stream().collect(Collectors.groupingBy(
                t -> new SP(t.getSubjectId(),
                    t.getPredicate()),
                Collectors.groupingBy(Triple::getObjectId)));

        List<Contradiction> found = new ArrayList<>();
        grouped.forEach((sp, byObject) -> {
            if (byObject.size() > 1) { // same S-P, different O
                List<Long> objs =
                    new ArrayList<>(byObject.keySet());
                for (int i = 0; i < objs.size(); i++)
                    for (int j = i + 1; j < objs.size(); j++) {
                        Triple a = byObject.get(objs.get(i)).get(0);
                        Triple b = byObject.get(objs.get(j)).get(0);
                        found.add(Contradiction.relation(
                            a, b, describe(a, b)));
                    }
            }
        });
        return found;
    }

    // Claim-level: same subject entity + opposing polarity +

```

```
// shingle-set similarity (Jaccard >= 0.5) -> contradiction.
public List<Contradiction> detectClaimLevel(
    List<Claim> judged) { /* ... */ }
}
```

Runs after every batch of approvals: new contradictions auto-populate the debate agenda.

ExtractionEngine — LLM proposals, human dispositions

```
@Service
public class ExtractionEngine {

    @Async("pipeline")
    public void runPipeline(Long documentId) {
        Document doc = docs.find(documentId);
        doc.setStatus(CHUNKED);           // 1. sentence-aware chunking
        List<DocumentChunk> chunks = chunker.chunk(doc);

        doc.setStatus(EXTRACTING);
        ExtractionRun run = runs.save(new ExtractionRun(doc));

        // 2. Per-chunk LLM extraction -> STRICT JSON -> DTO
        for (DocumentChunk c : chunks) {
            try {
                String raw = llm.complete(
                    PromptLibrary.extract(c.getContent()));
                ExtractionResultDto dto =
                    jsonExtractor.parse(raw);           // @Valid
                persistProposals(run, c, dto);           // PENDING / NEW
            } catch (JsonValidationException ex) {
                run.recordQuarantine(c, raw);           // never crash
            }
        }
        run.finish();
        doc.setStatus(AWAITING_APPROVAL);
        // NOTHING enters the knowledge base without the queue.
    }
}
```

The DTO that cages the AI (Unit 3's @Valid, weaponized):

```
public record ExtractionResultDto(
    @NotEmpty @Valid List<TripleDto> triples,
    @Valid @Size(max = 12) List<ClaimDto> claims) {

    public record TripleDto(
        @NotBlank @Size(max = 160) String subject,
        @NotBlank
        @Pattern(regexp = "[a-z_]{3,60}$") String predicate,
        @NotBlank @Size(max = 160) String object,
        @NotBlank String sentence) {}

    public record ClaimDto(
        @NotBlank String subject,
        @NotBlank @Size(max = 500) String claim,
        @NotNull
        @Pattern(regexp = "POSITIVE|NEGATIVE|NEUTRAL")
        String polarity,
        @NotBlank String sentence) {}
}
```

Entity resolution (deterministic v1): trim, uppercase, strip non-alphanumerics, collapse spaces; lookup-or-create on (normalized_name, type). Honest limitation, documented in the README; LLM-assisted matching is a stretch goal.

DebateEngine — the FSM with teeth

```
public enum DebateState { CREATED, ROUND_ACTIVE, AWAITING_CHAIR,
                          SYNTHESIZING, COMPLETED, ABORTED }
public enum DebateEvent { START, ARGUMENTS_DONE, WEIGHTS_SUBMITTED,
                          MAX_ROUNDS_REACHED, SYNTHESIS_DONE, ABORT }
```

```
@Service
public class DebateEngine {

    private static final int MAX_ROUNDS = 3;

    // Deterministic state machine: Java owns the debate skeleton.
    public void transition(Debate d, DebateEvent e) {
        switch (d.getState()) {
            case CREATED -> {
                if (e == START) startRound(d, 1); }
            case ROUND_ACTIVE -> {
                if (e == ARGUMENTS_DONE) {
                    d.setState(AWAITING_CHAIR);
                    bus.publish(d.getId(),
                               "ROUND_AWAITS_CHAIR"); } }
            case AWAITING_CHAIR -> {
                if (e == WEIGHTS_SUBMITTED) {
                    if (d.getCurrentRound() < MAX_ROUNDS)
                        startRound(d, d.getCurrentRound() + 1);
                    else beginSynthesis(d); } }
            case SYNTHESIZING -> {
                if (e == SYNTHESIS_DONE) {
                    d.setState(COMPLETED);
                    resolveContradiction(d); } }
            // ABORT allowed from any state
        }
    }

    // One round: three personas in PARALLEL (Unit 8 flex).
    @Async("llm")
    public void startRound(Debate d, int roundNumber) {
        d.setState(ROUND_ACTIVE);
        d.setCurrentRound(roundNumber);
        Round r = save(new Round(d, roundNumber, topicOf(d)));

        SkepticBrief brief =
            skepticAnalyst.prepare(d);           // machine evidence

        CompletableFuture<Argument> hawk =
            personaAgent.argue(HAWK, r, retrievalFor(HAWK, d));
        CompletableFuture<Argument> dove =
            personaAgent.argue(DOVE, r, retrievalFor(DOVE, d));
        CompletableFuture<Argument> skeptic =
            personaAgent.argue(SKEPTIC, r, brief);

        CompletableFuture.allOf(hawk, dove, skeptic).join();
        bus.publish(d.getId(), "ROUND_STREAMED");
        transition(d, ARGUMENTS_DONE);           // humans weight next
    }
}
```

SkepticAnalyst — the agent that wields Java

```
@Component
public class SkepticAnalyst {

    public SkepticBrief prepare(Debate d) {
```

```

// 1. Claims cited by Hawk/Dove in previous rounds
List<Claim> cited = citedClaimsOf(d);

// 2. Run the REAL verification engine on each
Map<Long, Verdict> liveVerdicts = cited.stream()
    .collect(Collectors.toMap(Claim::getId,
        verificationEngine::verify));

// 3. Query the REAL graph
Map<Long, Double> centrality =
    graphService.pageRankVerifiedOnly(d.corpusId());
//    -> "PageRank(X) = 0.31; top node is Y (0.58)"

return new SkepticBrief(liveVerdicts, centrality,
    graphService.communities(d.corpusId()));
// -> injected into the Skeptic's prompt as machine facts
}
}

```

DebateEventBus — SSE without WebFlux complexity

```

@Component
public class DebateEventBus {
    private final Map<Long, List<SseEmitter>> subs =
        new ConcurrentHashMap<>();

    public SseEmitter subscribe(Long debateId) {
        SseEmitter emitter = new SseEmitter(600_000L);
        subs.computeIfAbsent(debateId,
            k -> new CopyOnWriteArrayList<>()).add(emitter);
        emitter.onCompletion(() ->
            subs.get(debateId).remove(emitter));
        return emitter;
    }

    public void publish(Long debateId, String event,
        Object payload) {
        for (SseEmitter e : subs.getOrDefault(
            debateId, List.of())) {
            try {
                e.send(SseEmitter.event()
                    .name(event).data(payload));
            } catch (IOException ex) {
                subs.get(debateId).remove(e);
            }
        }
    }

    // events: "argument", "verdict", "round_complete",
    //         "awaiting_chair", "synthesis_ready"
}

```

9. The LLM Layer

PromptLibrary (the actual templates)

```

EXTRACT (cheap model, per chunk)
-----
You are an information-extraction engine. From the TEXT below,
extract exactly:
1. relations: subject, predicate, object -- predicate in
    snake_case from: reports_to, controls, funds, owns, works_for,
    located_in, allied_with, supplies, member_of (or new snake_case)
2. factual claims: subject, claim, polarity

```

```

(POSITIVE|NEGATIVE|NEUTRAL)

TEXT: ""{chunk}""

Return ONLY valid JSON:
{"triples":[{"subject":"","predicate":"","object":"","sentence":""}],
 "claims":[{"subject":"","claim":"","polarity":"","sentence":""}]}

Rules: extract ONLY what is explicitly stated; include the exact
source sentence for every item; no commentary.

JUDGE (mid model, per claim)
-----
You are a verification analyst.
CLAIM: {claim_text}
EVIDENCE PASSAGES (id, text): {retrieved_top5}
Decide: SUPPORTED (evidence states it), UNSUPPORTED (contradicted
or unrelated), EXAGGERATED (evidence supports only a weaker
version), SOURCE_MISSING (no relevant evidence).
Return ONLY JSON: {"verdict":"","confidence":0.0-1.0,
"reasoning":"","passage_ids":[]}

PERSONAS (best model, debate)
-----
HAWK: You argue the strong interpretation. Retrieve and cite
passages supporting decisive action on the question.
DOVE: You argue the cautious interpretation. Retrieve and cite
passages supporting restraint and risk.
SKEPTIC: You are armed with MACHINE EVIDENCE produced by
deterministic engines. Use every verdict and metric below
to demolish unsupported claims from both sides.
MACHINE EVIDENCE: {live_verdicts} {pagerank} {communities}
Quote it explicitly.
TOPIC: {contradiction description + both source sentences}

SYNTHESIS (after chair weights)
-----
You are the rapporteur. Write the final assessment of the debate.
Arguments with CHAIR WEIGHT (1-5, human judgment): {weighted_args}
Machine verdicts: {verdicts}
Structure: Executive summary / Hawk position / Dove position /
Skeptical findings / What the corpus establishes / What remains
uncertain / Recommendation with calibrated confidence language
(high/moderate/low confidence -- never invent certainty).

```

LlmClient — retry, timeout, provider-agnostic

```

@Component
public class LlmClient {
    private final WebClient web;    // baseUrl from LLM_BASE_URL env:
                                   // works with OpenAI, Groq, Ollama

    @Retryable(retryFor = LlmCallException.class,
               maxAttempts = 3,
               backoff = @Backoff(delay = 1000, multiplier = 2,
                                   maxDelay = 8000))
    public String complete(Prompt p) {
        return web.post().uri("/chat/completions")
            .bodyValue(request(p, p.maxTokens()))
            .retrieve().body(ChatResponse.class)
            .orTimeout(p.timeoutSec(), TimeUnit.SECONDS);
    }

    @Recover

```

```

    public String recover(LlmCallException e, Prompt p) {
        throw new PipelineStepFailed(
            "LLM unavailable after 3 attempts", e);
    }
}

```

Configuration (every knob external, cost capped)

```

prism:
  llm:
    base-url: ${LLM_BASE_URL:http://localhost:11434/v1}
    api-key:  ${LLM_API_KEY:local}
    extract-model: llama3.1:8b      # cheap
    judge-model:  llama3.1:8b      # mid
    debate-model:  ${DEBATE_MODEL:llama3.1:8b}
  limits:
    max-claims-per-doc: 12
    max-debate-rounds: 3
    persona-max-tokens: 512
    retrieval-top-k: 5
  trace:
    enabled: true
    detail-max-chars: 2000          # snapshots truncated, never unbounded
    retain-days: 30                 # @Scheduled cleanup

```

10. Frontend Design

```

src/
|-- api/
|   |-- axios.js           # instance + JWT interceptor + retry
|   |-- authApi.js documentsApi.js graphApi.js claimsApi.js
|   |-- debateApi.js chatApi.js traceApi.js adminApi.js
|-- auth/
|   |-- AuthContext.jsx    # user, token, login/logout
|   |-- ProtectedRoute.jsx # auth gate
|   |-- RoleGuard.jsx      # <RoleGuard roles=['VERIFIER']>
|-- components/
|   |-- Navbar.jsx StatCard.jsx DataTable.jsx
|   |-- Modal.jsx ConfirmDialog.jsx LoadingSpinner.jsx
|   |-- ErrorBoundary.jsx  # Unit 8
|   |-- RetryWrapper.jsx   # exponential backoff fetches
|   |-- trace/
|       |-- TraceTimeline.jsx TraceReplay.jsx TraceDag.jsx
|       |-- StepCard.jsx ThoughtRibbon.jsx useTraceStream.js
|-- hooks/
|   |-- useFetch.js usePagination.js useDebateStream.js
|-- pages/
|   |-- Dashboard/         # pipeline health stats
|   |-- Documents/         # list, upload, detail + pipeline
|   |-- ApprovalQueue/     # VERIFIER: pending triples
|   |-- GraphExplorer/     # * Cytoscape canvas
|   |-- Claims/            # verdict table + passages
|   |-- Contradictions/    # agenda -> "Convene Debate"
|   |-- DebateChamber/     # * live SSE, chair weighting
|   |-- Chat/              # grounded chat with citations
|   |-- Traces/            # * X-Ray browser + replay
|   |-- Admin/             # users, role management
|-- utils/

```

GraphExplorer — confidence-colored edges

```

const EDGE_COLORS = {
  VERIFIED: '#2ecc71', SINGLE_SOURCE: '#f39c12',
  CONTRADICTED: '#e74c3c' };

cytoscape({
  elements: [...nodes, ...edges.map(e => ({
    data: { id: `e${e.id}`, source: e.subjectId,
      target: e.objectId, label: e.predicate,
      color: EDGE_COLORS[e.confidence] },
  }))),
  style: [
    { selector: 'edge', style: {
      'line-color': 'data(color)',
      'target-arrow-shape': 'triangle',
      'curve-style': 'bezier' } },
    { selector: 'node', style: {
      'width': 'mapData(pagerank, 0, 0.6, 20, 70)',
      'height': 'mapData(pagerank, 0, 0.6, 20, 70)' } }
  ]
});
// toggle: [ All edges | Verified only ] -> re-fetch, re-render

```

useDebateStream — the SSE hook

```

export function useDebateStream(debateId) {
  const [args, setArgs] = useState([]);
  const [phase, setPhase] = useState('IDLE');
  useEffect(() => {
    const es = new EventSource(
      `/api/debates/${debateId}/stream`);
    es.addEventListener('argument', e =>
      setArgs(p => [...p, JSON.parse(e.data)]));
    es.addEventListener('verdict', e =>
      setArgs(p => attachVerdict(p, JSON.parse(e.data))));
    es.addEventListener('phase', e =>
      setPhase(JSON.parse(e.data).phase));
    return () => es.close();
  }, [debateId]);
  return { args, phase };
}

```

Chair weighting UI: each argument card has 1–5 dot buttons (POST `/api/debates/{id}/arguments/{argId}/w`) after all are weighted, the Advance/Synthesize button appears. The human is visibly inside the loop.

11. API Catalog

#	Endpoint	Method	Role gate	Unit
1	<code>/api/auth/register</code>	POST	public	3
2	<code>/api/auth/login</code> (issues JWT)	POST	public	5,6
3	<code>/api/auth/me</code>	GET	authed	5
4	<code>/api/documents?page&size&sort</code>	GET	ANALYST+	2,4
5	<code>/api/documents</code> (triggers async pipeline)	POST	ANALYST+	2,8
6	<code>/api/documents/{id}</code>	GET/DEL	ANALYST+/ADMIN2,4	
7	<code>/api/documents/{id}/pipeline</code> (re-run)	POST	ANALYST+	8
8	<code>/api/triples/pending?page&size</code>	GET	VERIFIER	4
9	<code>/api/triples/{id}/approve, /reject</code>	POST	VERIFIER	4
10	<code>/api/triples?status&page&sort</code>	GET	authed	4
11	<code>/api/entities?q&page&size</code>	GET	authed	4
12	<code>/api/entities/{id}</code> (dossier view)	GET	authed	4
13	<code>/api/graph?filter=ALL VERIFIED</code>	GET	authed	4
14	<code>/api/graph/centrality?mode</code>	GET	authed	8

#	Endpoint	Method	Role gate	Unit
15	/api/graph/communities	GET	authed	8
16	/api/claims?verdict&status&page	GET	authed	4
17	/api/claims/{id}/verify (async)	POST	ANALYST+	7,8
18	/api/claims/{id} (with passages)	GET	authed	4
19	/api/verdicts/{id}/adjudicate	POST	VERIFIER	5
20	/api/contradictions	GET	authed	4
21	/api/contradictions/{id}/debate	POST	VERIFIER	7
22	/api/debates/{id} (full state)	GET	authed	4
23	/api/debates/{id}/start (round 1)	POST	VERIFIER	8
24	/api/debates/{id}/stream	GET	authed	8
25	/api/debates/{id}/arguments/{arg}/weight	POST	VERIFIER	7
26	/api/debates/{id}/advance	POST	VERIFIER	7
27	/api/debates/{id}/report	GET	authed	4
28	/api/chat/sessions	POST/GET	authed	7
29	/api/chat/sessions/{id}/messages	POST	authed	7
30	/api/admin/users	GET/PATCH	ADMIN	5
31	/actuator/health, /metrics	GET	ADMIN	8

12. Core Workflows

W1 — Document ingestion (the pipeline)

1. ANALYST uploads doc (title + text)
2. POST /api/documents -> 201, status=UPLOADED, returns immediately
3. @Async("pipeline") fires:
 - a. Chunker: sentence-aware split, ~300 tokens/chunk
 - b. ExtractionRun created
 - c. per chunk: LlmClient.complete(EXTRACT prompt)
 - > JsonExtractor.parse -> @Valid ExtractionResultDto
 - > malformed? quarantine + continue
 - > valid? persist Triples(PENDING) + Claims(NEW)
 - > EntityService.resolve(): normalize, lookup-or-create
 - d. run stats saved -> doc status=AWAITING_APPROVAL
4. Frontend polls GET /api/documents/{id} -> progress bar
5. NOTHING is knowledge yet. Everything waits in the queue.

W2 — Approval and graph assembly

1. VERIFIER opens ApprovalQueue -> paginated PENDING triples, each showing S -P-> O, highlighted source sentence, doc title
2. Approve -> triple APPROVED (verified_by, decided_at)
3. Approval triggers (async):
 - a. ContradictionDetector.scan() -> new OPEN contradictions
 - b. @CacheEvict("centrality") -> graph metrics refresh
4. Reject -> triple REJECTED (kept for audit, off the graph)
5. GraphExplorer renders approved knowledge with confidence colors

W3 — Claim verification (Redline)

1. Claims listed (status NEW) -> "Verify all" (async batch)
2. Per claim:
 - a. FULLTEXT retrieve top-5 chunks
 - b. empty -> SOURCE_MISSING (no LLM cost, instant)
 - c. ClaimRuleEngine.assess() -> hedging penalty
 - d. LlmClient JUDGE prompt -> JSON verdict
 - e. fuse confidence = llm_confidence - hedging_penalty
rule override: weasel-heavy + absolutes -> EXAGGERATED
 - f. verdict + verdict_passages persisted -> claim JUDGED
3. Frontend: live-updating verdict table (poll or SSE)

4. CONTESTED verdicts -> VERIFIER adjudicates (human final say)

W4 — Debate round (Council, the crown)

```

1. VERIFIER picks an OPEN contradiction
  -> POST /api/contradictions/{id}/debate -> debate CREATED
2. POST /api/debates/{id}/start
  -> FSM: CREATED -> ROUND_ACTIVE, round 1 created
  -> SkepticAnalyst.prepare(): verify cited claims,
    pull PageRank + communities
  -> CompletableFuture: HAWK || DOVE || SKEPTIC (parallel)
  -> DebateEventBus streams: "argument", "verdict", "phase"
3. Chamber page renders arguments live; Skeptic arguments show
  UNSUPPORTED badges on demolished claims
4. FSM: ARGUMENTS_DONE -> AWAITING_CHAIR
5. Chair weights each argument 1-5 -> POST weight -> POST advance
  -> round < 3 ? startRound(r+1) : beginSynthesis
6. SYNTHESIZING: synthesis prompt (weighted args + verdicts)
  -> synthesis_reports row -> "synthesis_ready" SSE event
  -> FSM COMPLETED -> contradiction RESOLVED

```

W5 — Grounded chat

```

User: "Who is connected to Meridian Group and how reliable?"
1. FULLTEXT retrieve chunks + verified-triple lookup on entities
2. Prompt = SYSTEM(grounded rules) + chunks + relevant triples
  + verdict summary + graph metrics
3. Answer with citations: passages, triple IDs, verdicts.
  "Refuses what it cannot cite" is enforced by construction:
  the model only ever receives retrievable context.

```

W6 — The four-minute kill-shot demo script

```

0:00 Login as ANALYST -> upload corpus pack (30 pre-seeded memos)
0:20 Pipeline progress fills -> approval queue shows ~40 triples
0:40 Switch to VERIFIER -> approve a handful (fast clicks)
1:00 GraphExplorer: network blooms, edges colored by confidence,
  toggle Verified-only -> the network visibly re-shapes
1:30 Contradictions: "3 detected" -> Convene Debate -> Start
2:00 Chamber: Hawk/Dove/Skeptic argue LIVE (SSE); Skeptic kills
  claim 3 with a machine verdict badge
2:30 Chair: weight arguments -> Advance -> Synthesize
2:50 Report renders: markdown, cited, calibrated confidence
3:00 "But how do you know the machine is honest? Watch this."
3:10 Open X-Ray on Claim 41 -> press REPLAY
  -> retrieval, rules, LLM judgment, fusion math animate
3:30 Open the debate TraceDag -> scrub to SKEPTIC_BRIEF branch
3:50 Click a QUARANTINED extract step -> raw malformed LLM output
4:00 Silence in the room.

```

13. Glass Box: The Thought-Process Visualizer

The feature that turns PRISM from a system that works into a system that shows its work. In industry terms this is trace observability for AI execution — what LangSmith and Langfuse sell to ML engineers — rendered here as a first-class product feature. The viva sentence:

Most AI features are black boxes. I built a glass box — every decision the machine makes is recorded, timestamped, and replayable — including the moments where my Java code overrules the LLM.

Honest framing

A model's hidden chain-of-thought tokens cannot be rendered — they are inside the weights. What Glass Box renders is the *system's* reasoning: every input considered, every rule fired, every prompt sent, every response received, every override applied. That is more honest and more useful than a fake thinking spinner — and it is what production observability platforms actually do.

Design doctrine

Every operation emits a trace step. Three actors, three colors, one timeline:

Actor	Color	Represents
■ ENGINE	blue	deterministic Java: chunker, rules, retrieval, PageRank, FSM, contradiction scan
■ LLM	purple	probabilistic layer: extraction, judgment, personas, synthesis (prompt + response + tokens + latency captured)
■ HUMAN	gold	the verifier: approvals, adjudications, chair weights

Steps form a tree (parent/child), so parallel operations render as branches. Two modes: **LIVE** (trace events stream over the generalized SSE bus) and **REPLAY** (any completed run re-plays animated, step by step — the demo weapon).

Step-type taxonomy

```
ENGINE : CHUNK, ENTITY_RESOLVE, RULES, RETRIEVE, FUSION,
        CONTRADICTION_SCAN, PAGERANK, COMMUNITIES,
        FSM_TRANSITION, CACHE_HIT, CACHE_MISS
LLM    : EXTRACT, JUDGE, PERSONA_HAWK, PERSONA_DOVE,
        PERSONA_SKEPTIC, SKEPTIC_BRIEF, SYNTHESIZE, CHAT_ANSWER
HUMAN  : AWAITING_APPROVAL, APPROVE, REJECT, ADJUDICATE,
        CHAIR_WEIGHT
ERROR  : RETRY, QUARANTINE
```

TraceRecorder — one component, injected everywhere

Explicit recording (no AOP magic) keeps the code syllabus-honest and unit-testable. Persistence is `@Async` on a dedicated executor: tracing never slows the pipeline (under 5 ms added to the hot path — a Unit 8 talking point).

```
@Component
public class TraceRecorder {

    private final TraceRunRepository runs;
    private final TraceStepRepository steps;
    private final TraceEventBus bus;           // generalized SSE

    public TraceRunEntity begin(TraceKind kind, Long refId) {
        TraceRunEntity run =
            runs.save(TraceRunEntity.start(kind, refId));
        bus.publish(run.getId(), "run_started", run);
        return run;
    }

    /** Opens a step; close it later with the detail map. */
    public StepHandle step(TraceRunEntity run, String parentKey,
                          Actor actor, StepType type,
                          String title) {
        TraceStepEntity s = TraceStepEntity.open(
            run, parentOf(run, parentKey), nextSeq(run),
            actor, type, title);
        s.setStartedAt(Instant.now());
        return new StepHandle(s);
    }
}
```

```

/** Closes the step, persists ASYNC, broadcasts LIVE. */
@Async("trace")
public void close(StepHandle h, Map<String,Object> detail,
                  Status status, int tokens) {
    TraceStepEntity s = h.proto();
    s.setDurationMs((int) Duration.between(
        s.getStartedAt(), Instant.now()).toMillis());
    s.setDetail(trim(detail)); // capped at 2000 chars
    s.setStatus(status);
    s.setTokensUsed(tokens);
    steps.save(s);
    bus.publish(s.getRunId(), "step", s);
}

public void finish(TraceRunEntity run, String summary) {
    /* ... */
}
}

```

Retention job (another Unit 8 feature):

```

@Scheduled(cron = "0 0 3 * * *") // 3 AM daily
public void purgeOldTraces() {
    LocalDateTime cutoff =
        LocalDateTime.now().minusDays(retainDays);
    int deleted = traceStepRepository
        .deleteByRunStartedBefore(cutoff);
    log.info("trace-purge removed {} steps before {}",
        deleted, cutoff);
}

```

Integration — the traced verification (richest trace)

```

public Verdict verify(Claim claim, TraceRunEntity run) {
    StepHandle s1 = tracer.step(run, null, ENGINE,
        RETRIEVE, "FULLTEXT: \"" +
            truncate(claim.getClaimText(), 60) + "\"");
    List<DocumentChunk> evidence =
        retrieve(claim.getClaimText(), 5);
    tracer.close(s1, Map.of("hits", evidence.size(),
        "passages", previews(evidence)), OK, 0);

    StepHandle s2 = tracer.step(run, null, ENGINE,
        RULES, "Hedging rule scan");
    RuleAssessment ra = rules.assess(claim.getClaimText());
    tracer.close(s2, Map.of("weaselHits", ra.weaselHits(),
        "weaselWords", ra.words(), "absolutes",
        ra.absolutes(), "penalty",
        ra.hedgingPenalty()), OK, 0);

    if (evidence.isEmpty()) { // short-circuit, visible
        StepHandle s3 = tracer.step(run, null, ENGINE,
            FUSION, "SOURCE_MISSING - no LLM call");
        tracer.close(s3, Map.of(
            "savedCost", "1 LLM call", OK, 0);
        return Verdict.of(claim, SOURCE_MISSING, 0.9,
            "...", List.of());
    }

    StepHandle s4 = tracer.step(run, null, LLM,
        JUDGE, "LLM judgment");
    JudgeResponse jr = llm.judge(
        PromptLibrary.judge(claim, evidence));
    tracer.close(s4, Map.of("verdict", jr.verdict(),
        "confidence", jr.confidence(), "reasoning",

```

```

    jr.reasoning(), "promptSent", shortPrompt),
    OK, tokens);

StepHandle s5 = tracer.step(run, null, ENGINE, FUSION,
    "Fusion: " + jr.confidence() + " - "
    + ra.hedgingPenalty());
tracer.close(s5, Map.of("final", fused,
    "overrideApplied", overridden), OK, 0);
// ... return verdict
}

```

The Skeptic's evidence gathering renders as children under its `SKEPTIC_BRIEF` step — the room watches the agent arm itself before it speaks. Human steps are emitted from the approval and weighting endpoints, so gold dots appear in the timeline exactly where judgment happened.

Glass Box API surface

Endpoint	Method	What
/api/traces?kind&refId&page&size	GET	list runs (paged)
/api/traces/{runId}	GET	run + full step tree
/api/traces/{runId}/stream	GET	SSE live feed
/api/traces/{runId}/steps/{stepId}	GET	full step detail
/api/traces/stats	GET	calls, tokens, quarantine rate, latency

Deep-link convention: **Why?** buttons everywhere — every verdict card, graph edge, and argument carries an X-Ray link that jumps to the exact step, expanded.

The X-Ray experience — visual grammar

X-RAY - Document #12 "Q3 Internal Memos" - 47 steps - 4m 12s

```

-----
[E] CHUNK          "Chunking document"          84 chunks
    2ms #
[L] EXTRACT        chunk 1/84                    312 tokens
    1.4s #####
[L] EXTRACT        chunk 3/84  !! QUARANTINED
    malformed JSON -> raw response captured
...
[E] ENTITY_RESOLVE 132 names -> 96 entities (36 merges)
[E] CONTRADICTION_SCAN 3 found !!
[H] AWAITING_APPROVAL 41 triples -> verifier queue

```

Dot = actor color; bar length = duration (log scale); click any step to expand its detail JSON: retrieved passages, the prompt sent, the response, rule hits, override flags. The quarantined step, expanded, shows the raw malformed LLM output — proof the validation cage works.

TraceReplay — the animated playback

```

export function TraceReplay({ steps }) {
  const [i, setI] = useState(-1); // steps revealed so far
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(4); // 1x - 16x

  useEffect(() => {
    if (!playing || i >= steps.length - 1) return;
    // compress real duration, cap at 800ms: always snappy
    const dwell = Math.min(
      steps[i + 1].durationMs / speed, 800);
    const t = setTimeout(
      () => setI(x => x + 1), dwell);
    return () => clearTimeout(t);
  }, [playing, i, speed]);

  return (

```

```

    <div>
      <ReplayBar playing={playing}
        onPlay={() => setPlaying(true)}
        onPause={() => setPlaying(false)}
        speed={speed} onSpeed={setSpeed}
        progress={(i + 1) / steps.length}
        onScrub={setI} />
      <TraceTimeline steps={steps.slice(0, i + 1)}
        animate />
    </div>
  );
}

```

Press play on a claim verification and watch, in six seconds: retrieval hits → passages list → rule scan flags a weasel word → LLM judgment streams → fusion arithmetic appears (0.82 – 0.15 → 0.67) → if overridden, a red **RULE OVERRIDE: EXAGGERATED** flash.

TraceDag — parallelism made visible

```

          +-- [L] HAWK      1.3s -+
ROUND 2 --[E]-----+-- [L] DOVE    1.1s -+--> AWAITING [H] CHAIR
(FSM)      +-- [L] SKEPTIC  2.4s -+   (parallel -> 2.4s
      |           |                               wall, not 4.8s)
      +-- [E] SKEPTIC_BRIEF
          +-- [L] re-verify claim 3  UNSUPPORTED
          +-- [L] re-verify claim 7  SUPPORTED 0.9
          +-- [E] PAGERANK verified  X=0.31, Y=0.58

```

The Unit 8 parallelism claim stops being a README sentence and becomes a picture.

A claim's life, as the user sees it

```

Claim #41: "Meridian Group reportedly controls the entire
port authority"

[E] RETRIEVE  4 passages matched (doc 3 s2, doc 7 s1,
doc 12 s5, doc 12 s9)
[E] RULES     !! "reportedly" hit - no absolutes - penalty 0.15
[L] JUDGE     verdict SUPPORTED - conf 0.82 -
              "passage 3 states direct control of the authority
              board; passage 9 corroborates partially"
[E] FUSION    0.82 - 0.15 -> 0.67
FINAL: SUPPORTED @ 0.67 - evidence: 4 passages (clickable)

```

Build order (thin to fat, risk contained)

Tier	Effort	Contents
Thin	≈3 days	DDL + TraceRecorder + close() calls in VerificationEngine only + expandable list on ClaimDetail. Already demoable.
Medium	+2 days	pipeline + debate traces, ThoughtRibbon embeds, Why-buttons.
Fat	+2 days	TraceReplay animation, TraceDag, stats dashboard.

If time dies, the thin version still delivers the viva sentence. The replay animation is polish, not load-bearing — same doctrine as the debate chamber itself.

README one-liner

PRISM is a glass box: every retrieval, every rule, every prompt, every override, and every human judgment is recorded as a replayable trace — because an intelligence product you cannot audit is a product you cannot trust.

14. Security Design

```

register (BCrypt) -> login (AuthenticationManager)
-> JwtService.generate (claims: sub, role, iat, exp 24h,
    HS256, secret from env)
-> React stores token (localStorage v1; XSS trade-off
    documented in README)
-> axios interceptor: Authorization: Bearer <token>
-> JwtFilter validates signature + exp
-> SecurityContext (role -> authorities)
-> @PreAuthorize gates every write path

```

Permission matrix

Capability	ADMIN	ANALYST	VERIFIER
User management	✓	×	×
Upload documents / run pipeline	✓	✓	×
View graph / claims / contradictions	✓	✓	✓
Approve / reject triples	✓	×	✓
Adjudicate contested verdicts	✓	×	✓
Create / chair debates	✓	×	✓
Grounded chat	✓	✓	✓

Sample guards

```

@PreAuthorize("hasAnyRole('ADMIN','VERIFIER')")    // approve
@PreAuthorize("hasAnyRole('ADMIN','ANALYST')")      // upload
@PreAuthorize("hasRole('ADMIN')")                   // user mgmt

```

Environment variables: DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, LLM_BASE_URL, LLM_API_KEY, DEBATE_MODEL — never committed; .env.example ships in the repo instead.

15. Performance Layer

```

@Configuration
@EnableAsync @EnableCaching @EnableRetry
public class AsyncConfig {

    @Bean("pipeline")                // document ingestion
    public ThreadPoolTaskExecutor pipeline() {
        return pool(4, 8, 100, "prism-pipe-");
    }

    @Bean("llm")                    // persona/verification calls
    public ThreadPoolTaskExecutor llm() {
        return pool(6, 12, 50, "prism-llm-");
    }

    @Bean("trace")                  // Glass Box flushes
    public ThreadPoolTaskExecutor trace() {
        return pool(2, 4, 500, "prism-trace-");
    }
}

```

Concern	Mechanism	Where
Long jobs	@Async + status polling	ingestion, verify-all
Parallel LLM	CompletableFuture over personas / claims	debate rounds, verification batch
Expensive compute	@Cacheable centrality, communities (Caféine, TTL 10m)	graph metrics
Cache coherence	@CacheEvict(allEntries=true) on approval	approval flow
Flaky LLM	@Retryable, exponential backoff, 3 attempts	every LlmClient call
Observability	Logback structured lines + Actuator + trace stats	global
Frontend resilience	RetryWrapper, ErrorBoundary	api layer
Live UI	SSE (no polling storms)	chamber + traces
Trace overhead	async flush executor, kill-switch config	Glass Box
Retention	@Scheduled daily purge, 30-day window	Glass Box

16. Testing Strategy

Test class	Type	What it proves
GraphEngineTest	pure unit	PageRank on a hand-computed 4-node graph: <i>exact-value</i> assertions
GraphEngineCommunitiesTest	pure unit	two disjoint clusters separate by label propagation
ClaimRuleEngineTest	table-driven	“sources say X always” → weasel 1, absolutes true, penalty 0.25
ContradictionDetectorTest	pure unit	(A,reports_to,B)+(A,reports_to,C) → exactly 1 contradiction
DebateFsmTest	pure unit	illegal transitions rejected; happy path CREATED→COMPLETED
JsonExtractorTest	unit	malformed JSON → quarantine; predicate regex violation → reject
VerificationServiceTest	Mockito	SOURCE_MISSING short-circuit makes <i>zero</i> LLM calls
TraceRecorderTest	Mockito	step trees persist in order; detail truncation enforced
AuthFlowIT	MockMvc + Testcontainers	register → login → 403 without token → 200 with token → 403 wrong role
TripleApprovalIT	integration	approve → contradiction created + cache evicted
ChatRagIT	integration	mock LLM; assert retrieved chunks injected into prompt

Why Unit 9 becomes the easiest unit: the hardest components (graph math, rules, FSM) are deterministic pure Java — assert exact numbers, not ranges. CRUD apps do not get that luxury.

17. Docker and Deployment

```
# backend/Dockerfile -- multi-stage
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/prism-*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Frontend: node build → nginx serve with /api proxy.

```
# docker-compose.yml -- one command, whole system
services:
  mysql:
```

```

image: mysql:8
environment:
  MYSQL_DATABASE: prism
  MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
volumes: [ mysqldata:/var/lib/mysql ]
api:
  build: ./backend
  environment:
    DB_URL: jdbc:mysql://mysql:3306/prism
    JWT_SECRET: ${JWT_SECRET}
    LLM_BASE_URL: ${LLM_BASE_URL}
  depends_on: [ mysql ]
web:
  build: ./frontend
  ports: [ "3000:80" ]
  depends_on: [ api ]
volumes: { mysqldata: {} }

```

Unit 10 split: backend → Render/Railway (managed MySQL, env vars in dashboard); frontend → Vercel/Netlify (VITE_API_URL); Swagger via springdoc at `/swagger-ui.html`; README with the pipeline diagram, the deterministic-spine philosophy, honest limitations (entity resolution v1, LLM JSON failure rate, single-server SSE), and the demo GIF.

18. Build Plan and Phased Safety Net

Unit	Deliverable	Acceptance check (demo)
1	Scaffold; Navbar/Dashboard/StatCard; GET <code>/api/dashboard/stats</code>	dashboard renders live numbers from API
2	Documents CRUD end-to-end; layered backend; MySQL	create doc in UI → row appears → survives reload
3	Registration + validation; DTO, <code>@Valid</code> , <code>@ControllerAdvice</code>	bad password → 400 with per-field UI errors
4	Relation feast: entities/triples/claims; pagination, modals; manual entry + static Cytoscape view	hand-added triple appears on graph
5	Roles, AuthContext, ProtectedRoute, RoleGuard	VERIFIER sees queue tab, ANALYST does not
6	JWT end-to-end; interceptor; <code>@PreAuthorize</code>	analyst calls approve endpoint → 403
7	AI heart ON: extraction + approval queue + verification + contradictions + grounded chat	upload → approve → graph grows; chat answers with citations
8	Async pipeline + progress; parallel verification; caching + eviction; retry; SSE chamber; Glass Box thin tier	50-doc import does not block UI; debate streams live
9	Full test suite; Testcontainers ITs; Docker multi-stage + compose	<code>docker compose up</code> works; <code>mvn test</code> green
10	Deploy both halves; Swagger; README; 4-minute demo	kill-shot script runs on the deployed URL

PHASED SAFETY NET — the risk doctrine

Phase A (Units 1–4): corpus + manual knowledge base + graph view → demoable.

Phase B (Units 5–6): security lockdown → demoable.

Phase C (Unit 7): extraction + verification + contradictions → **at this point you already have a complete 10/10 project.** Everything after is flex.

Phase D (Unit 8): async + parallel + the debate chamber + Glass Box → the crown.

Phase E (Units 9–10): tests, Docker, deploy → ship it.

The safety line: if time dies anywhere, fall back to the last completed phase and still pass with a beast. Council is the stretch goal, not the load-bearing wall. Keep it documented as “Phase 2 roadmap” in the README — which still reads ambitious.

19. Risk Register

Risk	Likely	Impact	Mitigation (in design)
LLM returns malformed JSON	High	pipeline stalls	strict DTO validation → quarantine + continue, never crash
Entity resolution merges wrongly	Medium	ugly graph	deterministic v1 + VERIFIER queue catches it; documented limitation
Extraction quality poor on messy docs	Medium	weak demo corpus	curate a 25–30 doc synthetic corpus with seeded contradictions
LLM API cost at demo time	Medium	bill shock	Ollama for dev; caps in config; cheap model for extraction
SSE complexity	Medium	chamber bugs	bus is 40 lines; fallback = 2s polling (one-line swap)
Debate rounds exceed 60s	Medium	awkward demo	parallel personas (3× wall → 1×), token caps, 3-round max
Scope creep	High	unfinished	Unit-7 safety line; acceptance checks as hard gates
Trace storage growth	Medium	DB bloat	detail caps + 30-day scheduled purge
Tracing slows pipeline	Low	latency	async flush executor; <code>prism.trace.enabled</code> kill-switch
Detail JSON bloat in UI	Low	UI lag	server-side truncation + view-full on demand

20. What You Are Actually Building

Strip the skin and here is the truth of this document: a layered Spring Boot system in which a deterministic Java core (graph mathematics, forensic rules, a state machine, a contradiction detector, a trace recorder) governs a probabilistic LLM layer that is allowed to propose but never to decide — with a human verifier as the final authority — all persisted in a fully normalized relational model of nineteen tables, streamed to a React SPA over SSE, cached, retried, tested with exact-value assertions, containerized, deployed, and *replayable*: every retrieval, rule, prompt, override, and human judgment recorded as an auditable trace.

That is not a student project with an AI feature. That is an AI system with engineering discipline — which happens to score 10/10 on the training syllabus.

21. References

- Page, L., Brin, S., Motwani, R., Winograd, T., “The PageRank Citation Ranking: Bringing Order to the Web,” Stanford InfoLab Technical Report, 1999.
- Raghavan, U.N., Albert, R., Kumara, S., “Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks,” *Physical Review E*, 2007 (label propagation).
- Lewis, P., et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.

- Du, Y., et al., “Improving Factuality and Reasoning in Language Models through Multiagent Debate,” arXiv:2305.14325, 2023.
- Heuer, R. J., *Psychology of Intelligence Analysis*, CIA Center for the Study of Intelligence, 1999 (ACH).
- Clark, R. M., *Intelligence Analysis: A Target-Centric Approach*, CQ Press (Admiralty source grading).
- ICD 203, “Analytic Standards,” ODNI (public document).
- Jones, M., Bradley, J., Sakimura, N., RFC 7519: JSON Web Token (JWT), IETF, 2015.
- Provos, N., Mazières, M., “A Future-Adaptable Password Scheme” (bcrypt), *USENIX ATC*, 1999.
- Spring Boot: <https://spring.io/projects/spring-boot>
- Spring Security Reference: <https://docs.spring.io/spring-security/reference/>
- springdoc-openapi: <https://springdoc.org/>
- Testcontainers: <https://testcontainers.com/>
- Cytoscape.js: <https://js.cytoscape.org/>
- Vite: <https://vitejs.dev/> Docker: <https://docs.docker.com/>
- LangSmith (trace-observability context): <https://www.langchain.com/langsmith> Langfuse: <https://langfuse.com>

Reference verification notice

All cited works and tools are real, well-known publications and projects to the best of the compiler’s knowledge. Before formal academic submission, verify every venue, year, and identifier against DBLP, the IETF, or the publisher. This dossier is a starting map, not a camera-ready bibliography.